🚀 Gin Auth Project: The Ultimate Developer Guide

Welcome to the Gin Auth project! This repository contains a production-ready authentication system built with Go. It features JWT stateless auth, Real-world Email OTP, Mock SMS logging, and GORM database management.

🏗️ Integrated Technologies & Concepts

1. Gin Gonic: High-performance HTTP web framework.
2. JWT (JSON Web Tokens): Secure, stateless user authentication.
3. GORM: Modern ORM for Go mapping structs to MySQL tables.
4. Bcrypt: Industry-standard salted password hashing.
5. OTP (One-Time Password): Multi-factor security layer with 1-minute expiration.
6. Environment Configuration (.env): Securely managing secrets via godotenv.
7. SMTP Integration: Real-world communication via Gmail servers.
8. Middleware Pattern: Intercepting requests for authorization.
9. Graceful Degradation: Fallback mechanisms (Console logging) for third-party service failures.

🔄 The Full Flow: Step-by-Step Logic

1.User Registration (POST /register)

Input: name, email, phone, password, and send_to ("email" or "phone").

Logic:

- Duplicate Check: The system searches for existing users with the same email or phone.
- Unverified Handling: If a user exists but is unverified, the system allows re-registration (updating name/password) and triggers a New OTP. This acts as an OTP Resend mechanism.
- Hashing: Passwords (new or updated) are hashed using Bcrypt.
- OTP Generation: A new random 6-digit code is created.

- Expirations: The OTP record is updated/saved with a new 1-minute expiration.
- Delivery: OTP is sent via real Email or logged as a Mock SMS.
- OTP Verification (POST /verify-otp)

2. OTP Verification (POST /verify-otp)

Input: identifier (email or phone) and otp.

Logic:

- Lookup: The server searches for an OTP record matching the identifier and code.
- Expiry Check: The server checks if the current time is after the ExpiresAt timestamp.
- Verification: If valid, the user's is_verified flag is set to true.
- Cleanup: The OTP record is deleted to prevent reuse (One-Time use).
- User Login (POST /login)

3. User Login (POST /login)

Input: email and password.

Logic:

- Find User: Searches for the user by email.
- Security Check: If the user is found but is_verified is false, the login is rejected with a 403 Forbidden status.
- Password Check: Bcrypt compares the provided password with the stored hash.
- Token Issuance: If correct, a JWT is generated containing the user_id and an expiration time.

4. Accessing Protected Data (GET /api/profile)

Logic:

- Middleware intercept: The AuthMiddleware extracts the token from the Authorization header.
- Validation: The token signature is verified using the supersecretkey.
- Execution: If valid, the request proceeds to the handler; otherwise, it returns 401 Unauthorized.

🎓 Go Backend Interview Questions & Answers

🟢 Level 1: Basics (Gin & Go)

1. **What is a Gin Context (*gin.Context)?**
   It's the most important part of Gin. It carries request details, handles response writing, and allows you to pass variables between middleware and handlers.

2. **Why use gin.Default() instead of gin.New()?**
   Default() comes with Logger and Recovery (panic handling) middleware already attached. New() is completely blank.

3. **What is a Go Struct Tag?**
   They are strings like json:"email". They tell libraries (like Gin's JSON parser or GORM) how to map struct fields to external data formats.

🟡 Level 2: Intermediate (Authentication & Security)

1. **Explain the difference between JWT and Session-based auth.**
   Sessions are stateful (remembered by the server). JWT is stateless (the client holds the proof). JWT is better for horizontally scaling apps since the server doesn't need to check a database for every request.

2. **Why is password hashing (Bcrypt) better than encryption?**
   Encryption is reversible (two-way), meaning if a hacker gets the key, they get all passwords. Hashing is a one-way function. Even the developer cannot reverse a hash back into the original password.

3. **How do you handle expired JWTs?**
   The middleware checks the exp claim. If the current time is beyond exp, the token is invalid. In production, we often use Refresh Tokens to get a new access token without re-logging.

🔴 Level 3: Advanced (GORM & Database)

1. **What is an "Auto-Migration" in GORM?**
   It automatically updates your database schema to match your Go structs. It can add new columns and tables, but for safety, it never deletes or modifies existing columns.

2. **How would you improve the performance of a high-traffic OTP system?**
   I would use Redis (an in-memory store) instead of MySQL for storing OTPs. Redis has built-in TTL (Time-To-Live), so OTPs delete themselves automatically when they expire.

3. **What is an N+1 query problem, and how do you avoid it?**
   It happens when you fetch a list of items and then perform a separate query for each item's relationship. In GORM, we use .Preload() (Eager Loading) to fetch all data in 1 or 2 queries instead of N+1.

⛓ Level 4: Deep Dive (Database & GORM Methods)

1. **Explain the DSN (Data Source Name) string used in config/db.go.**
   The DSN user:pass@tcp(host:port)/dbname tells GORM which driver to use (MySQL), the credentials, the network protocol (TCP), the location of the server, and the specific database to target.

2. **How does db.First(&user) behave if the record is not found?**
   It returns a specific error: gorm.ErrRecordNotFound. In a web app, we should check for this error and return a 404 Not Found or 401 Unauthorized instead of a generic error.

3. **What is the difference between db.Create() and db.Save()?**
   Create() is strictly for inserting new rows. Save() is an "Upsert"—if the record has a Primary Key, it updates the existing row; otherwise, it inserts a new one.

4. **Why do we use config.DB.Where("email = ?").Delete(&models.OTP{}) in the Register flow?**
   To ensure Idempotency. If a user clicks Register multiple times, we delete any old, unverified OTPs for that email before creating a new one. This prevents the database from being flooded with expired codes.

5. **How does GORM prevent SQL Injection?**
   GORM uses Prepared Statements. When you use db.Where("email = ?", input), GORM sends the query and the data separately to the database. The database treats the input as a literal value, never as executable SQL code.

⚙️ Level 5: Environment & Configuration

1. **Why use godotenv.Load() instead of hardcoding API keys?**
   Hardcoding keys is a massive security risk. Using .env allows us to change keys for different environments (Dev, Prod) without recompiling the code. It also keeps secrets out of GitHub.

2. **How do you handle a missing .env file in production?**
   The code in config/db.go checks the error from godotenv.Load(). If it fails, it simply logs a warning and continues, allowing the app to use system environment variables (common in Docker/Kubernetes).

🩺 Level 6: Error Handling & Best Practices

1. **How do you handle registration attempts for accounts that are created but not yet verified?**
   Instead of blocking the user with an "already exists" error, we detect that the account is unverified. We then update their information (if they changed it) and trigger a New OTP. This provides a seamless "Resend OTP" experience without requiring a separate UI button.

2. **How do you handle Go errors in a Gin controller?**
   We use the "Check-and-Return" pattern. After every operation that can fail (like .Create()), we check if err != nil. If an error exists, we immediately send a JSON response with a relevant HTTP status code (like 400 or 500) and call return to stop the function.

3. **What is common practice for returning errors to the frontend?**
   We should return a structured JSON object, like gin.H{"error": "User-friendly message", "details": err.Error()}. The "details" field is helpful for developers but is often hidden in production for security.

4. **Why do we use defer resp.Body.Close() in the SMS/Email utilities?**
   To prevent Resource Leaks. HTTP response bodies must be closed to free up the network connection for reuse. defer ensures the body is closed even if the function exits early due to an error.

🔐 Level 7: Security Hardening

1. **How do you protect the /api/profile route?**
   Using Middleware. The AuthMiddleware verifies the JWT token before the request reaches the profile handler. If the token is missing or invalid, the request is Aborted with c.Abort().

2. **How would you prevent a user from reusing a verified OTP?**
   In our VerifyOTP logic, we call config.DB.Delete(&otp) immediately after successfully verifying the user. This ensures the code disappears from the database and cannot be used a second time.